



TEORÍA CLASE 4: MLOps en Kubernetes



Índice

- 1. [Introducción: De Docker Compose a Kubernetes](#)
- 2. [Conceptos Fundamentales de Kubernetes](#)
- 3. [Kubernetes Local: Kind vs Minikube](#)
- 4. [Kubernetes en la Nube](#)
- 5. [Infraestructure as Code con Terraform](#)
- 6. [Arquitectura de la Clase 4](#)
- 7. [¿Cuándo Usar Cada Solución?](#)



Introducción: De Docker Compose a Kubernetes

Evolución de la Arquitectura MLOps

CLASES 1–3: Docker Compose

- ✓ Perfecto para desarrollo local
- ✓ Fácil de entender y usar
- ✗ Limitado a un solo servidor
- ✗ No escala automáticamente
- ✗ No hay auto-recuperación ante fallos



CLASE 4: Kubernetes

- ✓ Escalamiento automático
- ✓ Auto-recuperación (self-healing)
- ✓ Distribución de carga
- ✓ Orquestación multi-nodo
- ✓ Producción-ready

¿Por Qué Kubernetes?

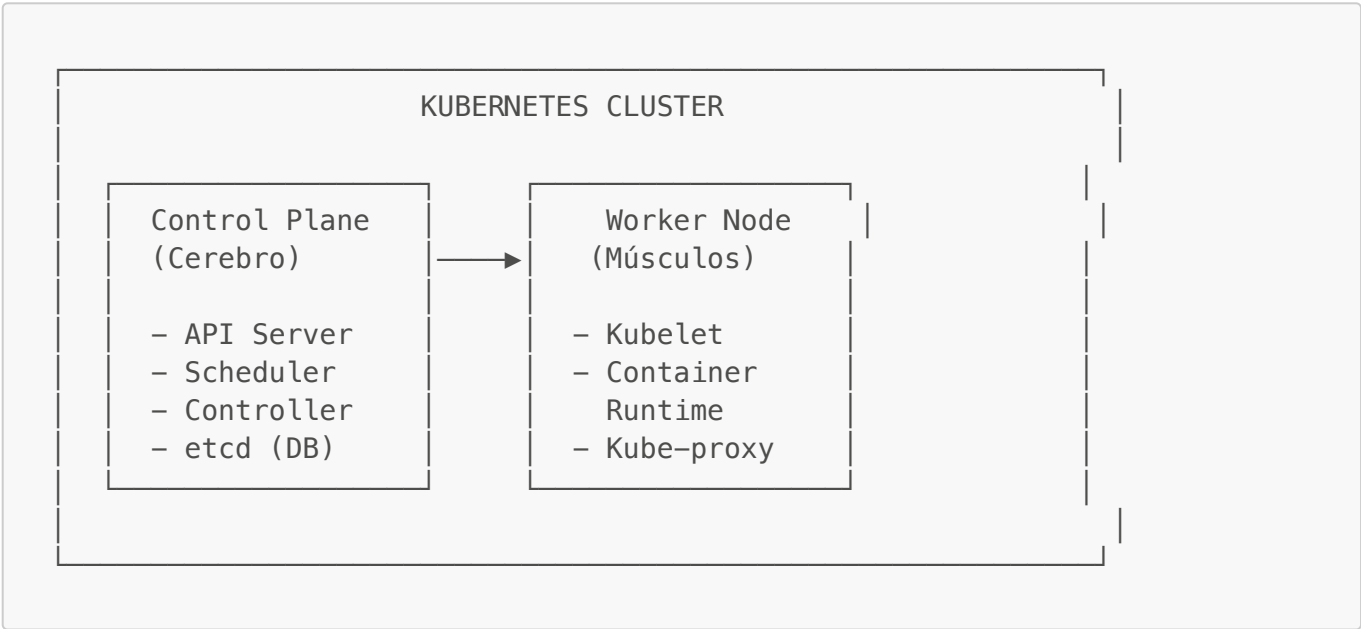
Característica	Docker Compose	Kubernetes
Escalamiento	Manual	Automático (HPA)
Alta Disponibilidad	✗ No	✓ Sí
Auto-recuperación	✗ No	✓ Sí (self-healing)
Múltiples nodos	✗ No	✓ Sí

Característica	Docker Compose	Kubernetes
Load Balancing	Básico	Avanzado
Rolling Updates	✗ No	✓ Sí (zero downtime)
Service Discovery	DNS básico	DNS + Service mesh
Configuración	.yaml simple	Más complejo, más potente
Uso en Producción	Dev/Test	Producción

 Conceptos Fundamentales de Kubernetes

1. Cluster (Clúster)

El **Cluster** es el conjunto completo de recursos de Kubernetes.

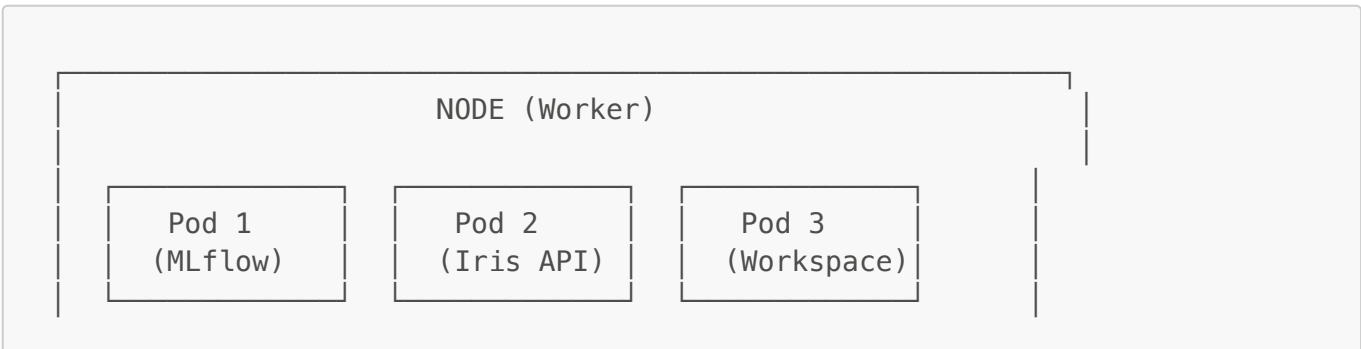


Componentes:

- **Control Plane:** Gestiona el cluster (toma decisiones)
- **Worker Nodes:** Ejecutan las aplicaciones (contenedores)

2. Node (Nodo)

Un **Node** es una máquina (física o virtual) en el cluster.



Recursos del Nodo:

- CPU: 4 cores
- Memoria: 8 GB RAM
- Disco: 100 GB
- IP: 10.244.0.1

Tipos de Nodos:

- **Control Plane Node:** Gestiona el cluster (no corre aplicaciones por defecto)
- **Worker Node:** Ejecuta los pods de las aplicaciones

 En Producción:

- Típicamente: 3+ Control Plane Nodes (alta disponibilidad)
- Típicamente: 5-50+ Worker Nodes (dependiendo de la carga)

 En Nuestra Clase:

- 1 Control Plane Node
- 2 Worker Nodes (para demos de distribución)

3. Pod

Un **Pod** es la unidad más pequeña de Kubernetes. Contiene uno o más contenedores.

POD**Contenedor Principal**

Aplicación: MLflow Server
Puerto: 5000
Recursos: 512MB RAM, 0.5 CPU

Metadata del Pod:

- Nombre: mlflow-86c56bcf7b-2vwp9
- IP Interna: 10.244.2.2
- Nodo: worker-2
- Estado: Running

Características:

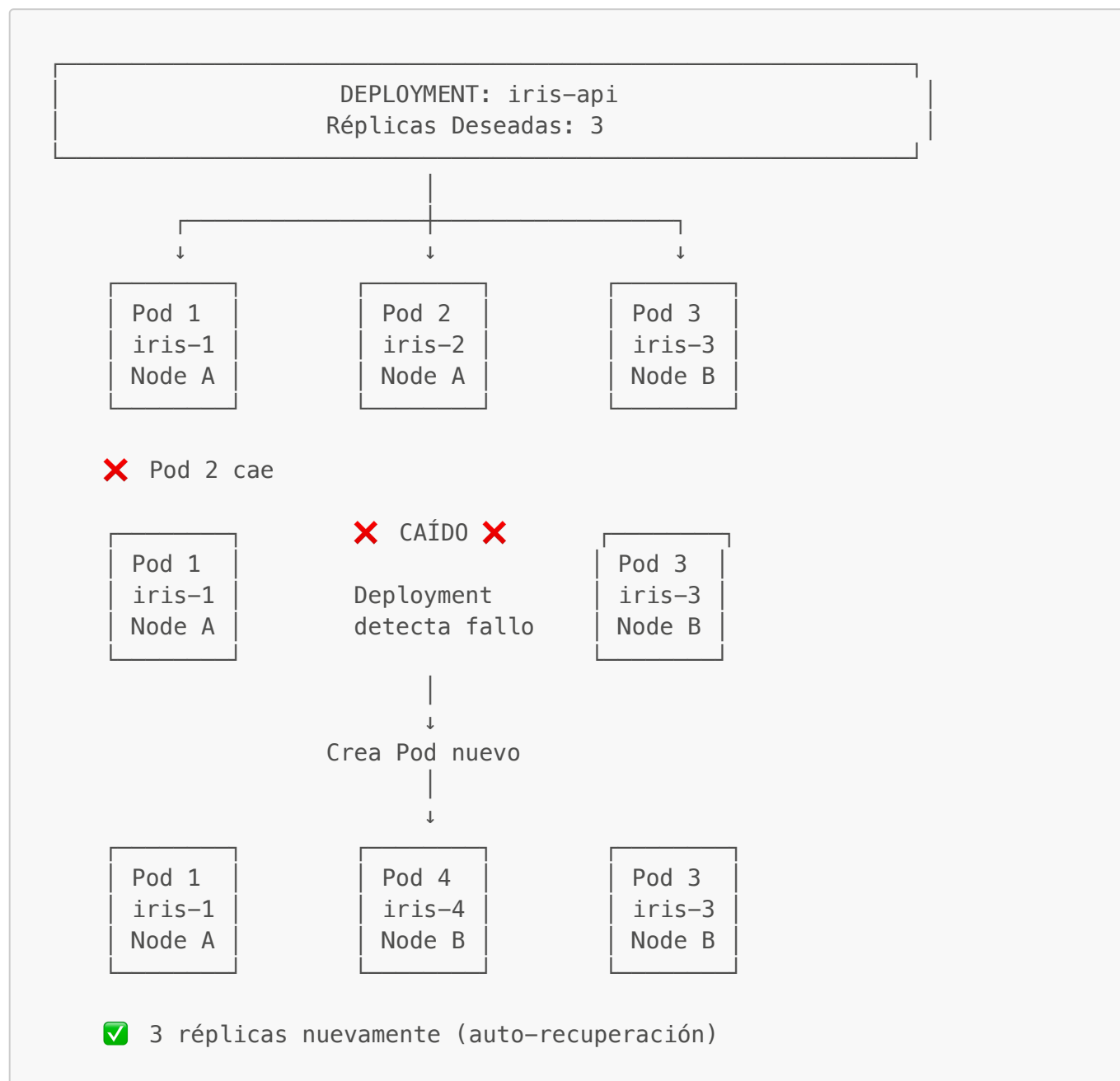
- Tiene una IP única dentro del cluster
- Comparte namespace de red entre contenedores

- Es efímero (puede ser recreado en cualquier momento)
- Kubernetes gestiona su ciclo de vida automáticamente

 **Regla General:** 1 Pod = 1 Contenedor (en la mayoría de casos)

4. Deployment

Un **Deployment** gestiona réplicas de Pods y garantiza el estado deseado.



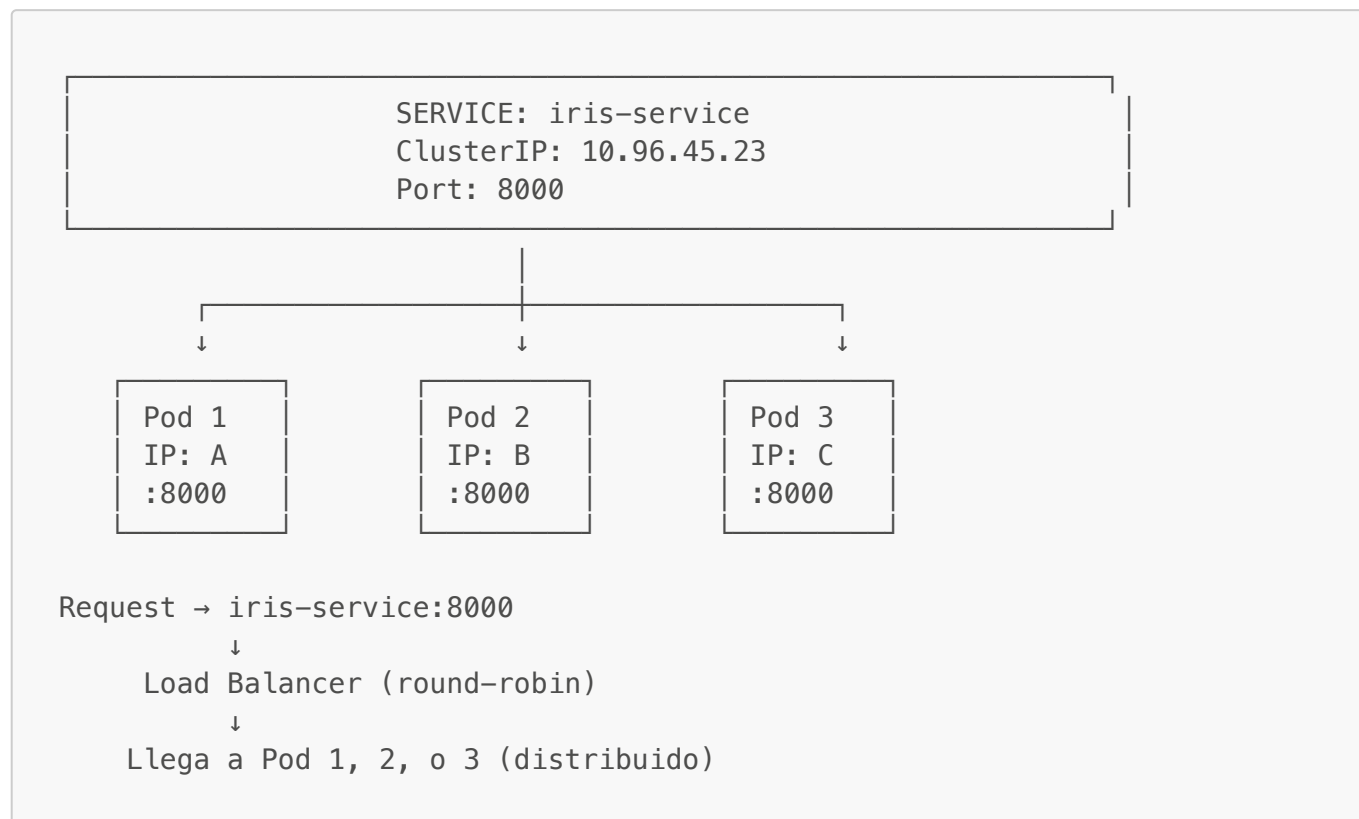
Responsabilidades del Deployment:

- ✅ Mantener el número deseado de réplicas
- ✅ Recrear pods que fallan (self-healing)
- ✅ Rolling updates (actualizaciones sin downtime)
- ✅ Rollback a versiones anteriores
- ✅ Escalar hacia arriba o abajo

💡 **Deployment = Declarativo:** Le dices "quiero 3 réplicas", Kubernetes se encarga del "cómo"

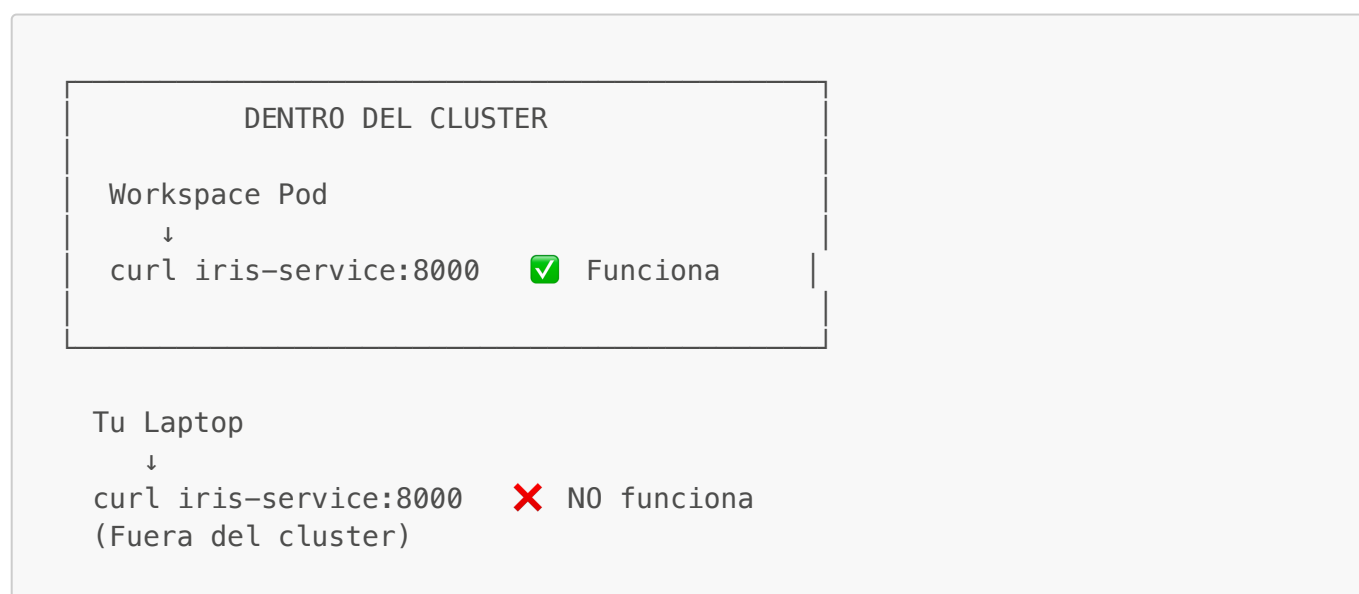
5. Service

Un **Service** expone los Pods y balancea la carga entre ellos.



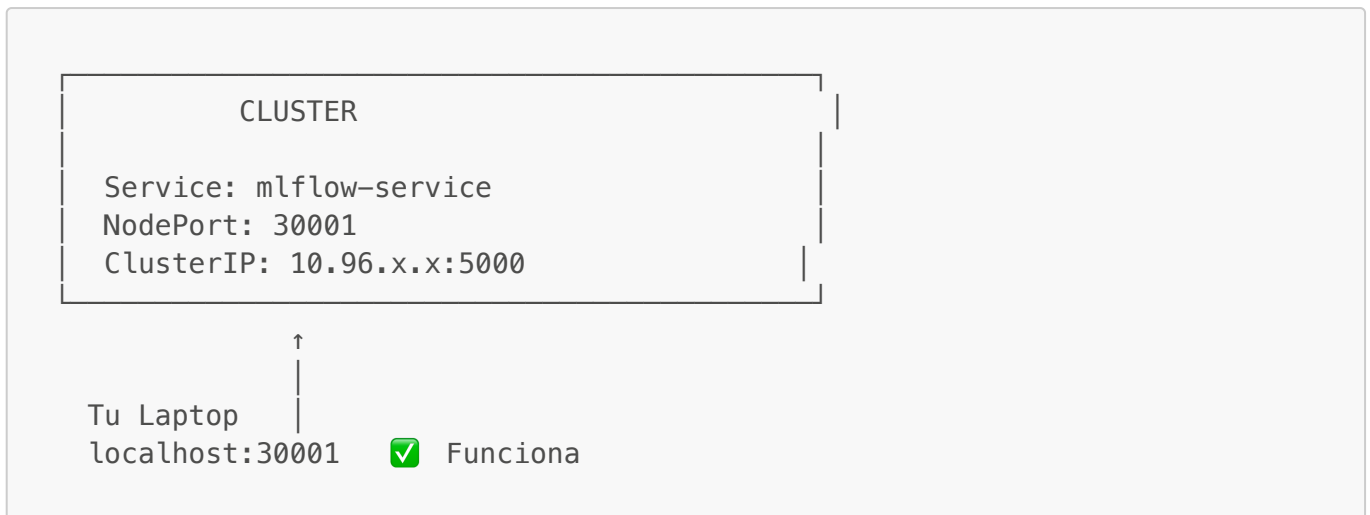
Tipos de Services:

A. ClusterIP (interno)



Uso: Comunicación interna entre servicios (la más común)

B. NodePort (externo)

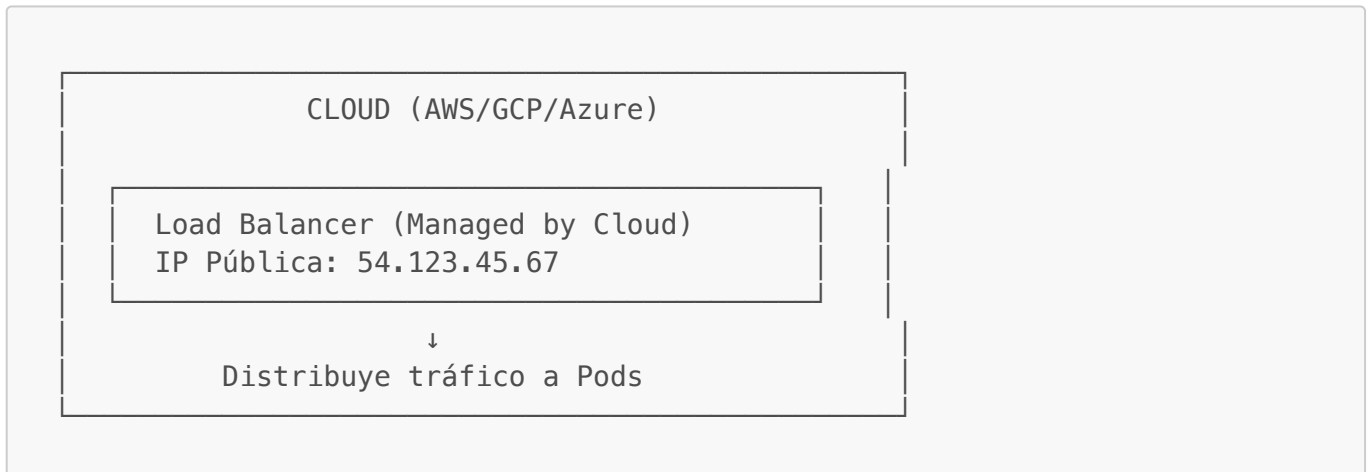


Uso: Acceso desde fuera del cluster (desarrollo/testing)

Puertos en nuestra clase:

- MLflow: 30001
- Evidently: 30002
- Jupyter: 30003
- Iris API: 30004

C. LoadBalancer (producción en cloud)

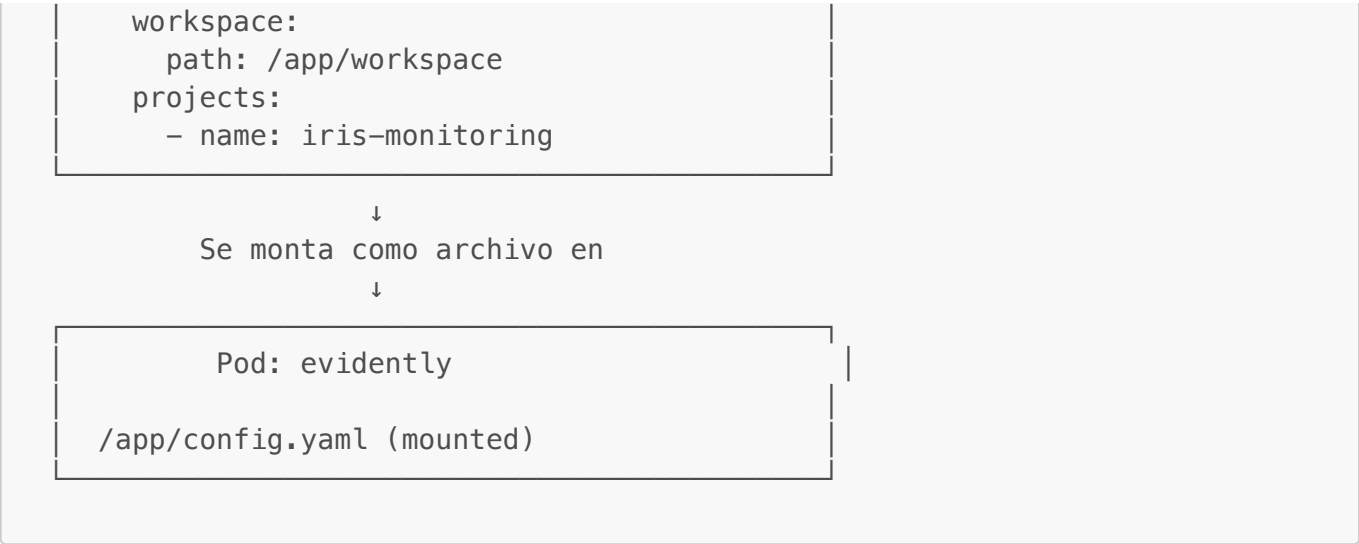


Uso: Producción en la nube (AWS ELB, GCP Load Balancer, Azure LB)

6. ConfigMap

Un **ConfigMap** almacena configuración no sensible como key-value pairs.





Uso:

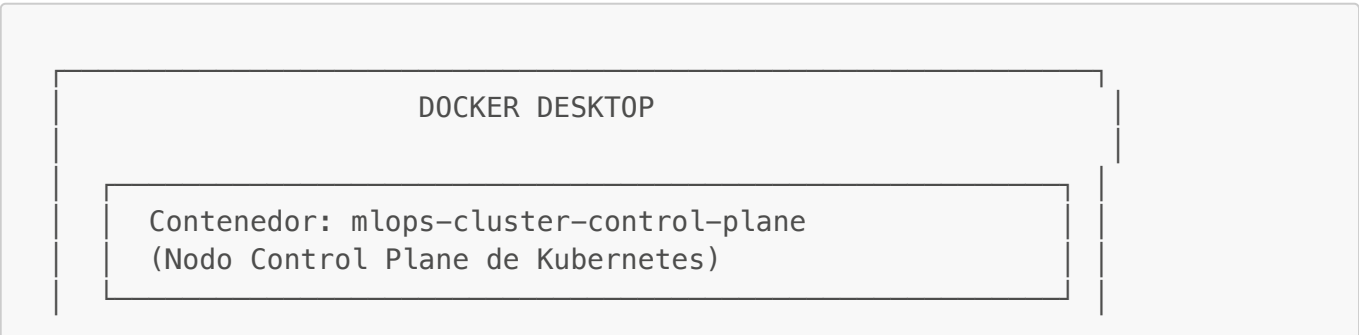
- Configuración de aplicaciones
- Variables de entorno
- Archivos de configuración

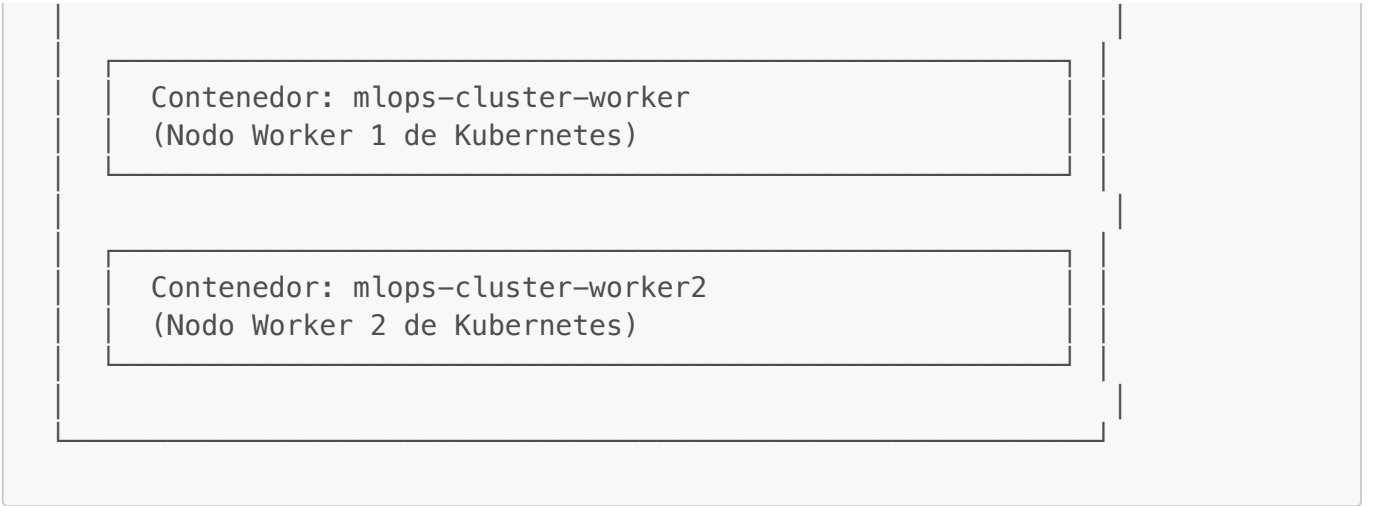
Kubernetes Local: Kind vs Minikube

Comparación de Soluciones Locales

Característica	Kind (nuestra elección)	Minikube	Docker Desktop K8s
Tecnología	Kubernetes en Docker	VM o Docker	Kubernetes integrado
Velocidad	⚡ Muy rápido	🐢 Más lento (VM)	⚡ Rápido
Uso de recursos	💚 Bajo	🟡 Medio-Alto	💚 Bajo
Multi-nodo	✅ Fácil	✅ Posible	❌ Solo 1 nodo
Complejidad	🟢 Simple	🟡 Media	🟢 Muy simple
Producción-like	✅ Alto	✅ Alto	🟡 Medio
Instalación	CLI simple	CLI + Driver	Incluido con Docker
Casos de uso	Testing, CI/CD	Desarrollo local	Desarrollo simple

Kind (Kubernetes in Docker)





```
Contenedor: mlops-cluster-worker  
(Nodo Worker 1 de Kubernetes)
```

```
Contenedor: mlops-cluster-worker2  
(Nodo Worker 2 de Kubernetes)
```

✅ Ventajas de Kind:

- Rápido de iniciar (~30 segundos)
- Fácil de crear múltiples clusters
- Ideal para testing y CI/CD
- Configuración con YAML simple
- Usado por Kubernetes SIG Testing

❌ Limitaciones:

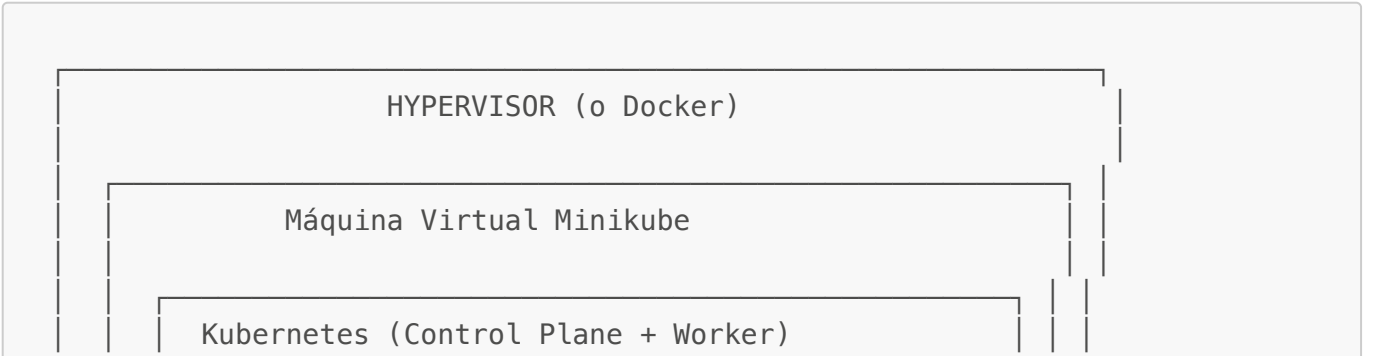
- Solo para desarrollo/testing
- No tiene UI gráfica
- Networking más complejo que Minikube



Instalación:

```
# macOS  
brew install kind  
  
# Windows  
choco install kind -y --ignore-dependencies  
  
# Crear cluster  
kind create cluster --name mlops-cluster
```

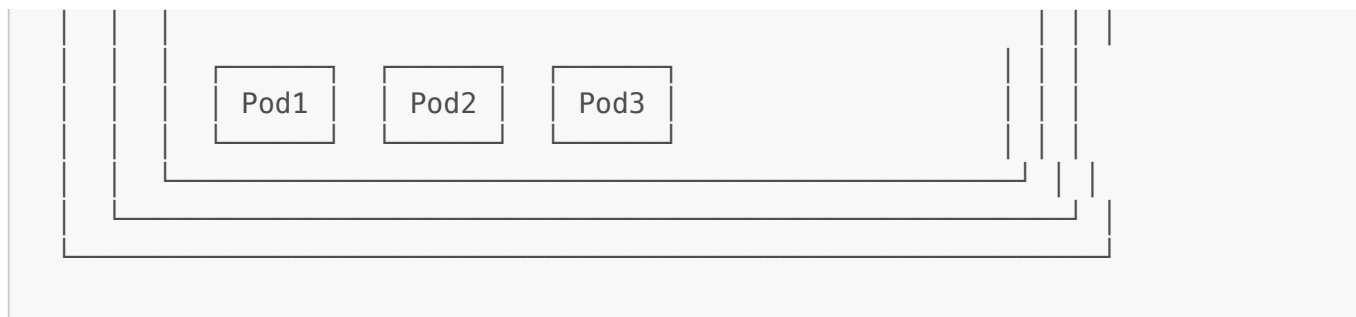
Minikube



```
HYPERVISOR (o Docker)
```

```
Máquina Virtual Minikube
```

```
Kubernetes (Control Plane + Worker)
```



✓ Ventajas de Minikube:

- Más maduro y establecido
- Soporta addons (dashboard, metrics-server, ingress)
- UI integrada: [minikube dashboard](#)
- Múltiples drivers (Docker, VirtualBox, HyperKit)
- Excelente documentación

✗ Limitaciones:

- Más lento para iniciar (~1-2 minutos)
- Consume más recursos (VM)
- Multi-nodo más complejo



Instalación:

```
# macOS
brew install minikube

# Windows
choco install minikube -y

# Iniciar
minikube start --driver=docker
```

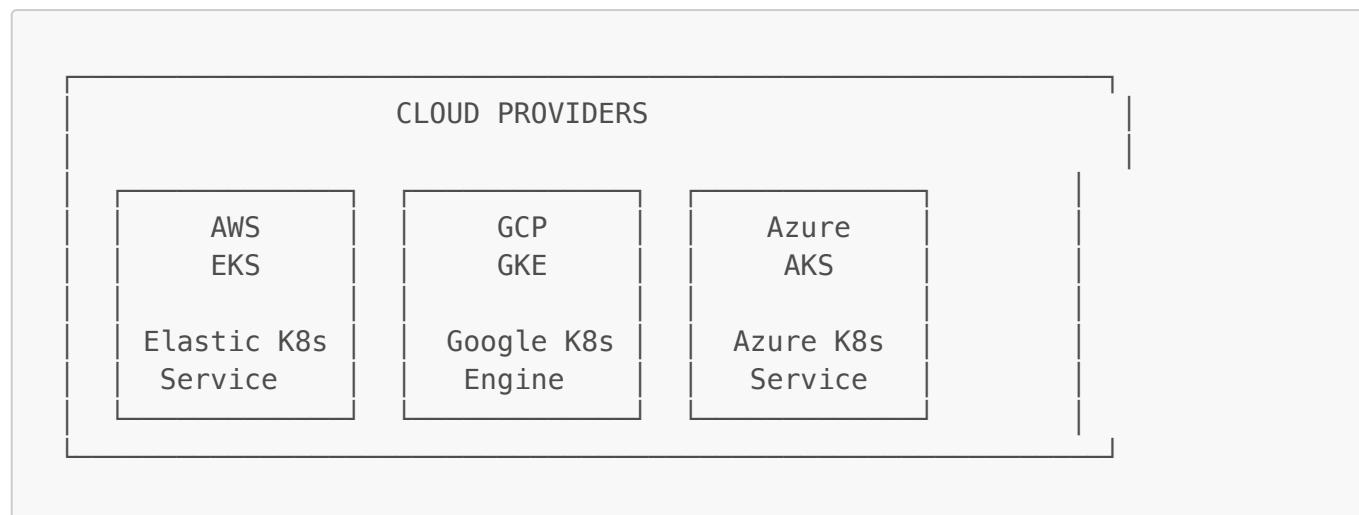
¿Por Qué Elegimos Kind para esta Clase?

Criterio	Razón
Velocidad	Clusters en 30 segundos (vs 2 min de Minikube)
Multi-nodo	Fácil de demostrar distribución de pods
CI/CD-like	Similar a lo que se usa en producción para testing
Recursos	Usa menos RAM y CPU
Simplicidad	No requiere configurar drivers



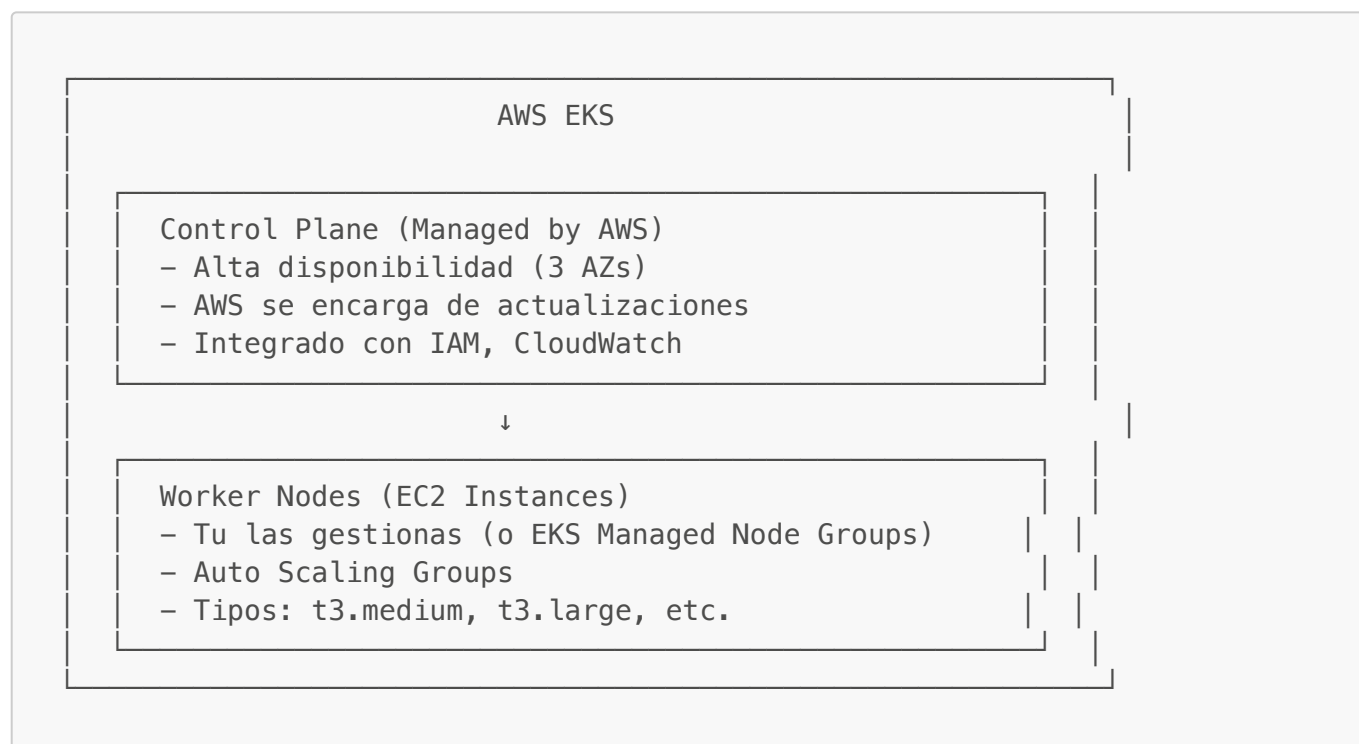
Kubernetes en la Nube

Opciones Managed Kubernetes



1. Amazon EKS (Elastic Kubernetes Service)

Arquitectura:



Características:

- Integración profunda con servicios AWS (RDS, S3, ALB)
- Soporte para Fargate (serverless containers)
- Certificado Kubernetes conformance
- Costo: \$0.10/hora por cluster + EC2 instances

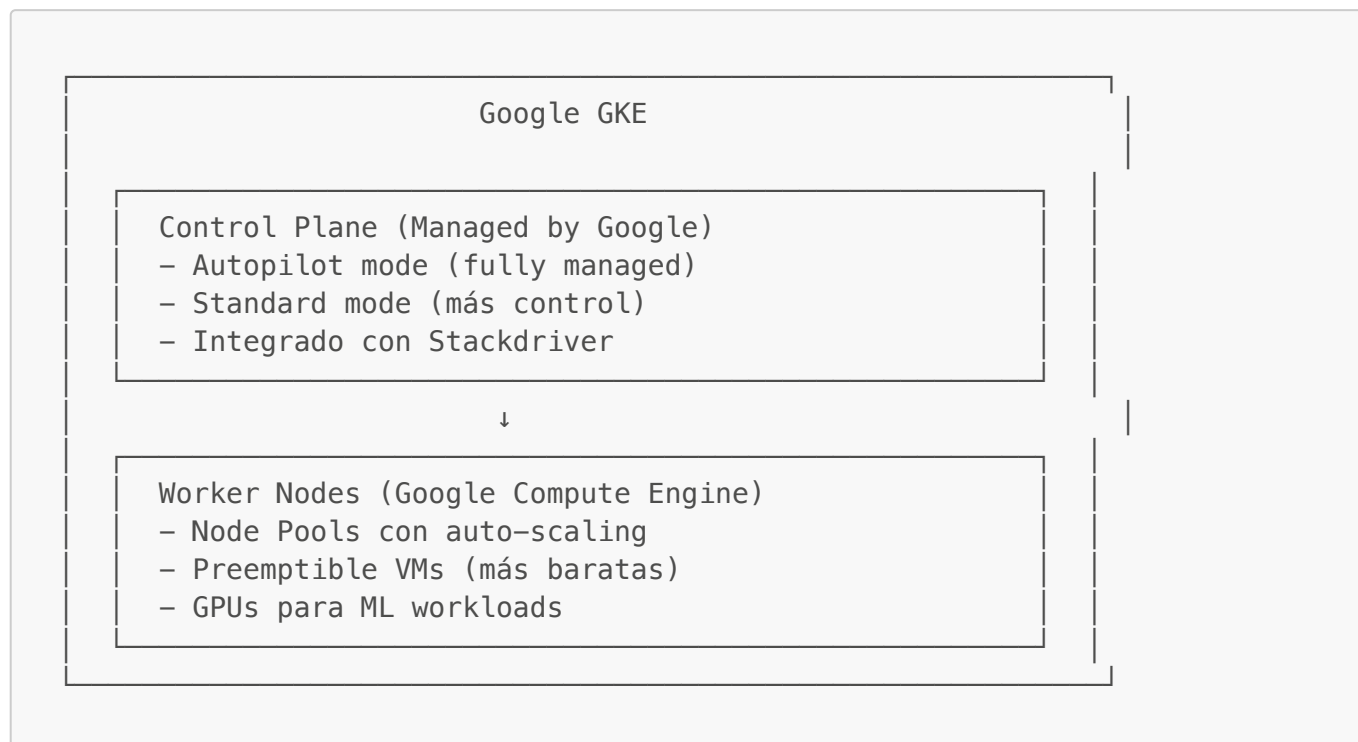
Cuándo usar:

- Ya usas AWS
- Necesitas integración con RDS, S3, etc.

- Equipos grandes con presupuesto

2. Google GKE (Google Kubernetes Engine)

Arquitectura:



Características:

- ✅ Kubernetes fue creado por Google (experiencia profunda)
- ✅ Autopilot mode (Google gestiona todo)
- ✅ Mejor precio/rendimiento
- ✅ Ideal para workloads de ML (soporte para TPUs)
- 💰 Costo: Gratis para Control Plane + GCE instances

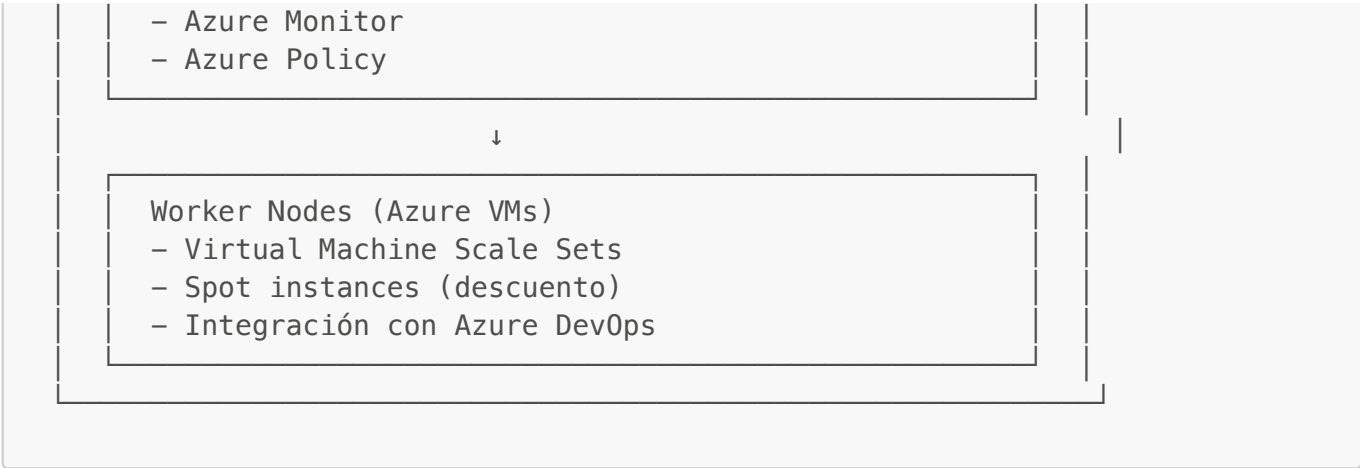
Cuándo usar:

- Startup/equipo pequeño
- Workloads de ML/AI intensivos
- Quieres menos gestión operativa

3. Azure AKS (Azure Kubernetes Service)

Arquitectura:





Características:

- Integración con ecosistema Microsoft
- Gratis el Control Plane
- Azure DevOps + GitHub Actions
- Buena integración con Windows containers
- Costo: Gratis para Control Plane + Azure VMs

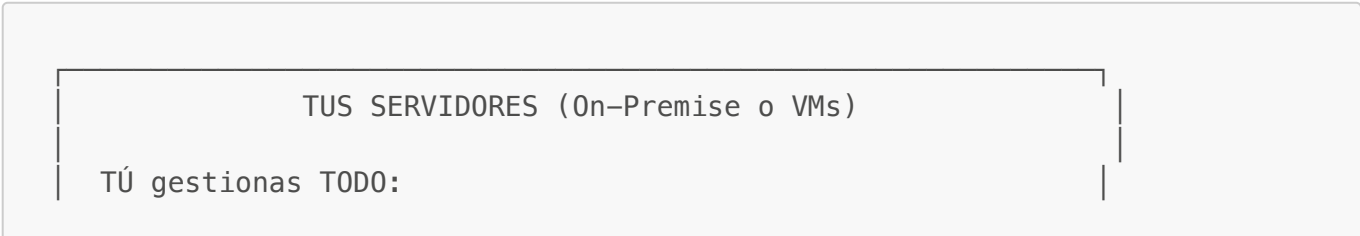
Cuándo usar:

- Empresa usa Microsoft Azure
- Stack .NET o Windows containers
- Integración con Azure DevOps

Comparación Cloud Providers

Aspecto	AWS EKS	Google GKE	Azure AKS
Madurez K8s	Buena	Excelente (creó K8s)	Buena
Facilidad de uso	Media	Fácil	Media
Precio Control Plane	\$0.10/hr	Gratis	Gratis
ML/AI Support	Bueno	Excelente (TPU)	Bueno
Autoescalado	Sí	Sí (mejor)	Sí
Serverless Pods	Fargate	GKE Autopilot	Virtual Nodes
Marketshare	33%	26%	20%

Kubernetes Self-Managed (No Recomendado para Empezar)



- ✗ Instalación de Kubernetes
- ✗ Actualizaciones del Control Plane
- ✗ Certificados SSL
- ✗ etcd backups
- ✗ Networking (CNI)
- ✗ Storage (CSI)
- ✗ Monitoring y Logging
- ✗ Security patches

Cuándo usar self-managed:

- Requisitos de compliance muy estrictos
- Datos no pueden salir del data center
- Tienes un equipo DevOps/SRE grande
- Control total sobre cada aspecto

💡 **Recomendación:** Empieza con Managed Kubernetes (EKS/GKE/AKS)

🏗️ Infrastructure as Code con Terraform

¿Qué es Infrastructure as Code (IaC)?

ANTES: Manual (Imperativo)

1. Abrir AWS Console
2. Crear VPC → Clic, clic, clic...
3. Crear Subnets → Clic, clic, clic...
4. Crear EC2 → Clic, clic, clic...
5. Configurar Security Groups → Clic...

- ✗ No reproducible
- ✗ Propenso a errores
- ✗ No versionable
- ✗ Difícil de replicar

↓

AHORA: Terraform (Declarativo)

```
archivo.tf:

resource "kubernetes_deployment" "mlflow" {
  replicas = 1
  ...
}

$ terraform apply
```

- ✓ Reproducible (mismo código = mismo resultado)
- ✓ Versionable (Git)
- ✓ Declarativo (describe QUÉ, no CÓMO)
- ✓ Documentación automática

Terraform Workflow

TERRAFORM WORKFLOW

1. WRITE
 - └─ Escribes .tf files
 - └─ Describe infraestructura deseada
2. PLAN
 - └─ \$ terraform plan
 - └─ Terraform calcula diferencias
 - └─ Muestra QUÉ va a cambiar
3. APPLY
 - └─ \$ terraform apply
 - └─ Ejecuta cambios
 - └─ Guarda estado en terraform.tfstate
4. MANAGE
 - └─ Modifica .tf files
 - └─ Vuelve a plan + apply
 - └─ Terraform mantiene sincronización

Ejemplo: Deployment con Terraform

```
# Imperativo (kubectl - manual)
$ kubectl create deployment mlflow --image=mlflow:latest
$ kubectl scale deployment mlflow --replicas=2
$ kubectl expose deployment mlflow --port=5000

# Declarativo (Terraform - código)
resource "kubernetes_deployment" "mlflow" {
  metadata {
    name = "mlflow"
  }

  spec {
    replicas = 2
  }
}
```

```

    template {
      spec {
        container {
          name = "mlflow"
          image = "mlflow:latest"
          port {
            container_port = 5000
          }
        }
      }
    }
  }
}

resource "kubernetes_service" "mlflow" {
  metadata {
    name = "mlflow-service"
  }

  spec {
    type = "NodePort"

    selector = {
      app = "mlflow"
    }

    port {
      port          = 5000
      target_port    = 5000
      node_port      = 30001
    }
  }
}
```

Ventajas:

-  Todo en Git (historial completo)
-  Code reviews de infraestructura
-  CI/CD para infraestructura
-  Rollback fácil (git revert + terraform apply)

Terraform vs Otras Herramientas

Herramienta	Lenguaje	Cloud Support	Kubernetes	Estado
Terraform	HCL	Todos	 Excelente	Remote state
Pulumi	Python/TS/Go	Todos	 Excelente	Cloud backend
CloudFormation	YAML/JSON	Solo AWS	 Limitado	AWS managed
ARM Templates	JSON	Solo Azure	 Limitado	Azure managed

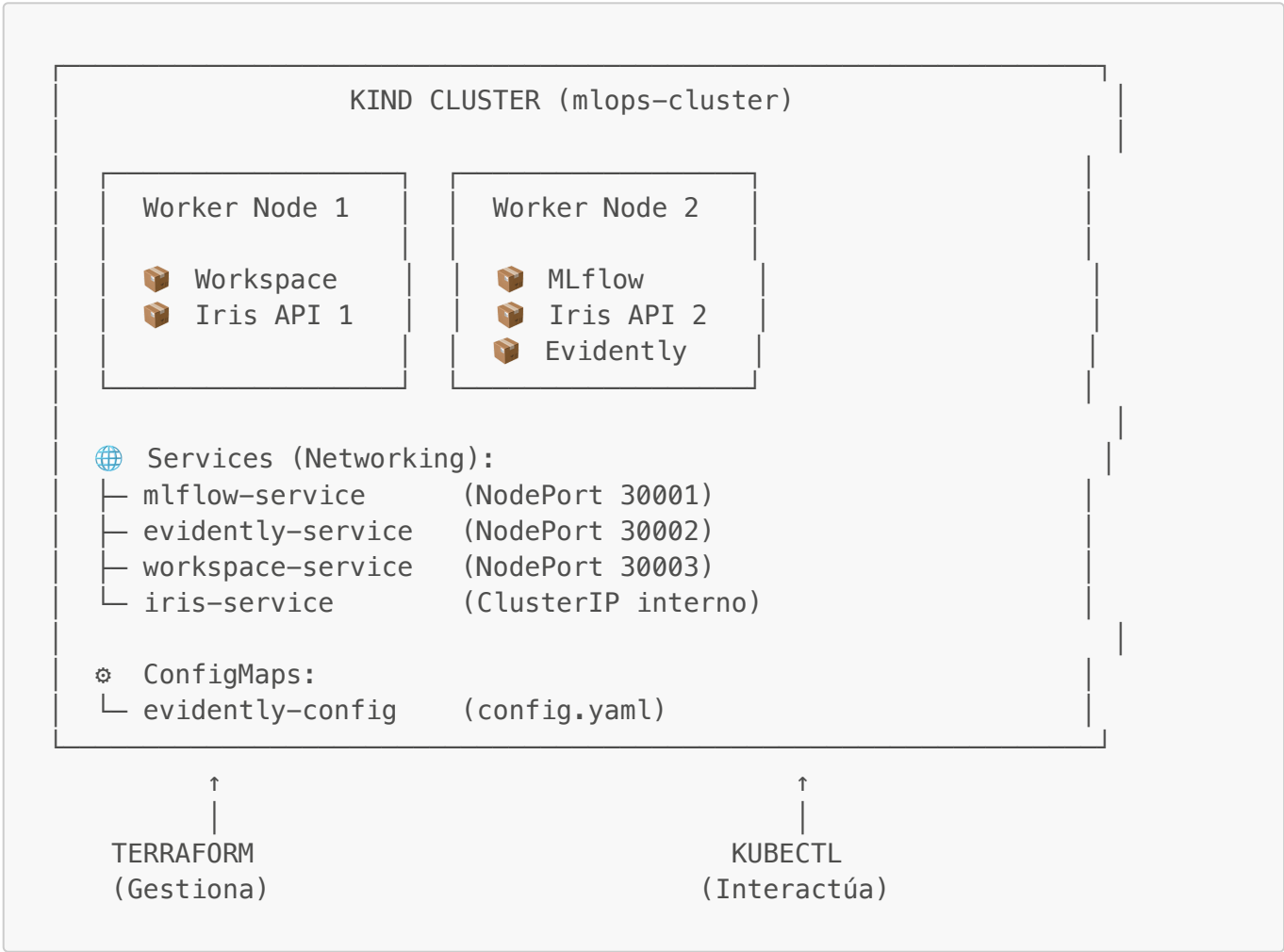
Herramienta	Lenguaje	Cloud Support	Kubernetes	Estado
Ansible	YAML	Todos	 Bueno	No idempotente

 Por qué Terraform:

- Multi-cloud (no te ata a un proveedor)
- Gran ecosistema de providers
- HCL es más legible que JSON
- Estado centralizado

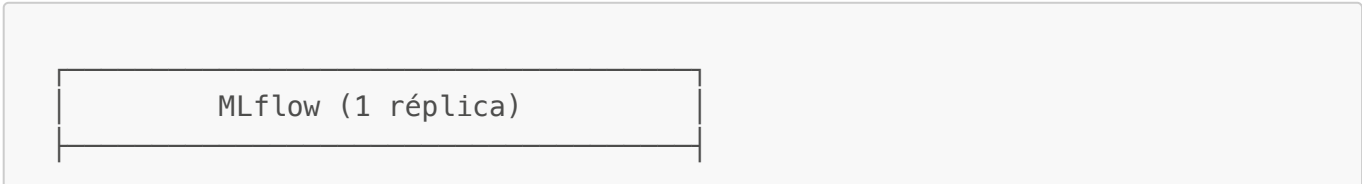
 Arquitectura de la Clase 4

Vista General



Componentes de Nuestra Arquitectura

1. MLflow Server



Función: Tracking de experimentos
Puerto: 5000 → NodePort 30001
Acceso: http://localhost:30001
Almacenamiento: SQLite (dev)
Uso: Registrar métricas, modelos

2. Evidently Service

Evidently (1 réplica)

Función: Detección de drift
Puerto: 8000 → NodePort 30002
Acceso: http://localhost:30002
ConfigMap: config.yaml
Uso: Monitoreo de calidad de datos

3. Iris API

Iris API (2 réplicas)

Función: Inferencia de ML
Puerto: 8000 (ClusterIP)
Acceso: Interno + NodePort 30004
Modelo: Cocinado en imagen
Load Balancing: Automático

4. Jupyter Workspace

Workspace (1 réplica)

Función: Entorno de desarrollo
Puerto: 8888 → NodePort 30003
Acceso: http://localhost:30003
Librerías: pandas, mlflow, etc.
Notebooks: Copiados con kubect1

Flujo de Trabajo en la Clase

1. CONSTRUCCIÓN
 - └─ docker build app_iris
 - └─ docker build app_workspace
 - └─ kind load docker-image (cargar al cluster)
2. DESPLIEGUE
 - └─ kind create cluster
 - └─ terraform init
 - └─ terraform plan (revisar)
 - └─ terraform apply (desplegar)
3. CONFIGURACIÓN
 - └─ kubectl cp notebooks (copiar al workspace)
4. USO
 - └─ Abrir Jupyter (localhost:30003)
 - └─ Ejecutar notebook 01_simulacion.ipynb
 - └─ Ver resultados en MLflow (localhost:30001)
 - └─ Ver drift en Evidently (localhost:30002)
5. GESTIÓN
 - └─ Escalar: terraform (cambiar replicas)
 - └─ Ver logs: kubectl logs
 - └─ Debuggear: kubectl describe
 - └─ Limpiar: terraform destroy

Patrón Inmutable: Modelo "Cocinado"

PATRÓN TRADICIONAL

Dockerfile:

```
COPY app.py /app/  
CMD ["python", "app.py"]
```

Modelo: Se carga desde volumen externo

❌ Requiere sincronización

❌ Volúmenes compartidos complejos

↓

PATRÓN INMUTABLE (Nuestra clase)

Dockerfile:

```
COPY train.py /app/  
RUN python train.py  
COPY app.py /app/
```

← Entrena durante BUILD

```
CMD ["python", "app.py"]
```

Modelo: Incluido en la imagen (model.joblib)

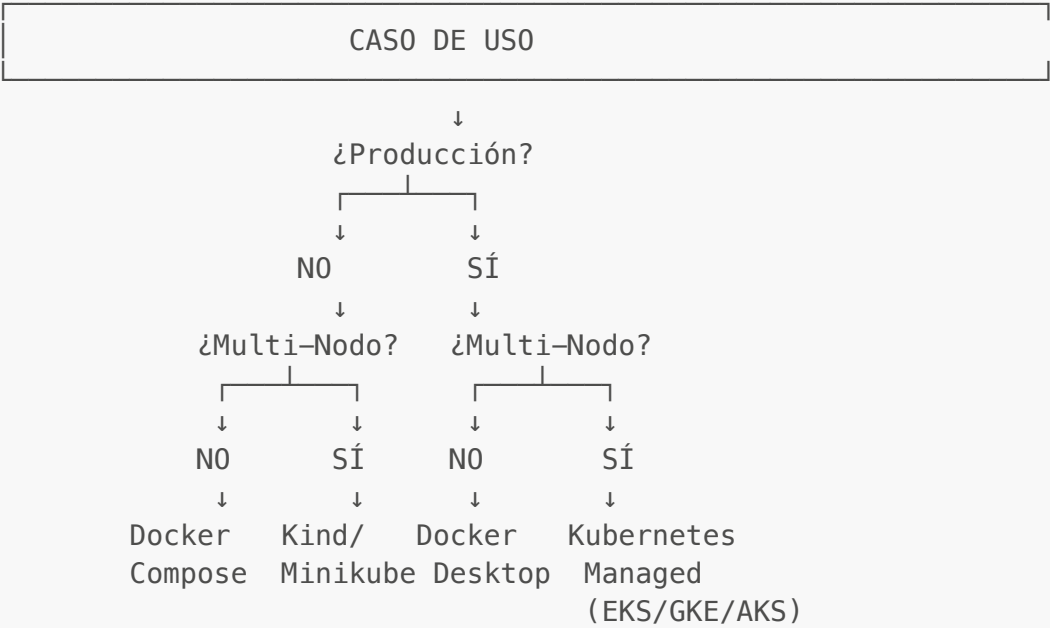
- ✓ Auto-contenido
- ✓ Reproducible
- ✓ Versionable (tags de imagen)

Ventajas del Patrón Inmutable:

- ✓ La imagen contiene TODO lo necesario
- ✓ No dependencias externas en runtime
- ✓ Fácil rollback (cambiar tag de imagen)
- ✓ Testing determinístico

¿Cuándo Usar Cada Solución?

Matriz de Decisión



Recomendaciones por Escenario

Escenario	Solución Recomendada	Por Qué
Desarrollo Local (1 servicio)	Docker run	Simple, rápido
Desarrollo Local (2-5 servicios)	Docker Compose	Fácil de configurar
Aprendizaje de K8s	Kind o Minikube	Kubernetes real, local
Testing CI/CD	Kind	Rápido, ligero, scriptable

Escenario	Solución Recomendada	Por Qué
Producción pequeña	GKE Autopilot	Managed, sin gestión
Producción media	EKS/GKE/AKS Standard	Control + managed
Producción grande	EKS/GKE/AKS Multi-cluster	Escala, geo-redundancia
Compliance estricto	Kubernetes On-Premise	Control total

Progresión de Aprendizaje Recomendada

1. Docker (Clase 1)

- └ Contenedores básicos

2. Docker Compose (Clases 2-3)

- └ Orquestación simple local

3. Kubernetes Local (Clase 4) ← ESTAMOS AQUÍ

- └ Kind + Terraform
- └ Conceptos K8s fundamentales

4. Kubernetes Managed (Clase 5+)

- └ EKS/GKE/AKS
- └ Ingress, HPA, Monitoring

5. Kubernetes Avanzado (Opcional)

- └ Service Mesh (Istio)
- └ GitOps (ArgoCD)
- └ Multi-cluster

 Resumen Ejecutivo

Conceptos Clave de Hoy:

Concepto	Definición	Analogía
Cluster	Conjunto de nodos de K8s	Fábrica completa
Node	Máquina (física/virtual)	Planta de la fábrica
Pod	Unidad mínima, contenedores	Máquina individual
Deployment	Gestiona réplicas de Pods	Capataz de producción
Service	Load balancer para Pods	Recepcionista/dispatcher
ConfigMap	Configuración	Manuales de operación

De Docker Compose a Kubernetes:

Aspecto	Docker Compose	Kubernetes
Complejidad	🟢 Baja	🟡 Media
Escalabilidad	🔴 Limitada	🟢 Excelente
Producción	❌ No recomendado	✅ Diseñado para ello
Aprendizaje	🟢 Rápido	🟡 Más tiempo

Kind vs Minikube vs Cloud:

Solución	Velocidad	Costo	Producción
Kind	⚡ ⚡ ⚡	🆓	❌
Minikube	⚡ ⚡	🆓	❌
EKS/GKE/AKS	⚡	💰 💰	✅

Terraform:

Declarativo + Versionable + Reproducible = IaC

📚 Recursos Adicionales

Documentación Oficial:

- Kubernetes: <https://kubernetes.io/docs/>
- Kind: <https://kind.sigs.k8s.io/>
- Terraform: <https://registry.terraform.io/providers/hashicorp/kubernetes/>

Tutoriales Interactivos:

- Kubernetes Basics: <https://kubernetes.io/docs/tutorials/kubernetes-basics/>
- Play with Kubernetes: <https://labs.play-with-k8s.com/>

Libros Recomendados:

- "Kubernetes Up & Running" - Kelsey Hightower
- "Terraform: Up & Running" - Yevgeniy Brikman

🎓 ¿Listo para la Práctica?

Ahora que entiendes la teoría:

1. ✅ Sabes qué es un Pod, Node, Deployment, Service
2. ✅ Entiendes por qué Kubernetes vs Docker Compose

3. ☒ Conoces las opciones: Kind, Minikube, Cloud
4. ☒ Comprendes Infrastructure as Code

 **Es hora de ponerlo en práctica con Kind + Terraform!**

→ **Continúa con: ONBOARDING.md**
